Introduction

primUR is a pipeline for the automated discovery of taxon-specific diagnostic PCR primers. Given a list of target organisms, it identifies unique genomic regions (URs) absent from related neighbor genomes, designs primers against those regions, and tests their *in silico* specificity. The pipeline integrates six steps:

1. Taxon lookup and neighbor discovery via `taxi` and `neighbors`
2. Genome download via the NCBI `datasets` CLI
3. Phylogenetic tree construction via `phylonium`, `nj`, `midRoot`, `land`, and `plotTree`
4. Unique region discovery via `makeFurDb`, `fur`, and `cleanSeq`
5. Primer design via `fa2prim`, `primer3_core`, and `prim2tab`
6. *In silico* specificity testing via `scop`

The program is invoked as

```
$ primUR -t query.txt -n neidb -d scopdb [options]
```

where `query.txt` contains one target species per line, `neidb` is the path to the neighbor genome database, and `scopdb` is the path to a BLAST nucleotide database for specificity testing.

Installation

Clone the repository and build the binary with

```
$ git clone https://github.com/lloyd-noll/primUR
$ cd primUR
$ make
```

Install all pipeline dependencies with

```
$ make install
```

Then install the `primUR` binary to `~/go/bin` with

```
$ make install-bin
```

Make sure `~/go/bin` is in your `PATH`. If not, add the following line to your `~/.zshrc` or `~/.bashrc` and reload your shell:

```
export PATH=$PATH:~/go/bin
```

Verify all dependencies are available with

```
$ make check
```

Implementation

The outline of `primUR` contains hooks for imports, types, variables, functions, and the logic of the main function.

Program: `primUR`

⟨*primUR.go*⟩≡
```
package main
import (
  ⟨Imports⟩
)
⟨Types⟩
⟨Variables⟩
⟨Functions⟩
func main() {
  ⟨Main function⟩
}
```

In the main function we parse arguments, check dependencies, read the query file, set up the log directory, and then run the pipeline for each target species.

⟨*Main function*⟩≡
```
cfg := parseArgs()
if err := checkDependencies(requiredTools); err != nil {
  log.Fatal(err)
}
targets, err := readLines(cfg.queryFile)
if err != nil {
  log.Fatalf("Could not read query file: %v", err)
}
if err := setupLogDir(); err != nil {
  log.Fatalf("Could not create log directory: %v", err)
}
for _, target := range targets {
  if err := processTarget(target, cfg); err != nil {
    log.Printf("Error processing %q: %v", target, err)
  }
}
```

The imports cover standard library packages for buffered I/O, command line flag parsing, formatted output, logging, file system access, external process execution, path manipulation, regular expressions, and string handling.

⟨*Imports*⟩≡
```
"bufio"
"flag"
"fmt"
"log"
"os"
"os/exec"
"path/filepath"
"regexp"
"strings"
```

Types and Variables

The `config` struct holds all values parsed from the command line.

⟨*Types*⟩≡
```
type config struct {
  queryFile string
  neiDB     string
  scopDB    string
  verbose   bool
  checkOnly bool
  resume    bool
}
```

The variable `requiredTools` lists all external binaries that must be present in `PATH` before the pipeline starts.

⟨*Variables*⟩≡
```
var requiredTools = []string{
  "taxi", "neighbors", "datasets", "unzip",
  "phylonium", "nj", "midRoot", "land", "plotTree",
  "makeFurDb", "fur", "cleanSeq", "cres", "fa2prim",
  "primer3_core", "prim2tab", "scop",
}
```

Helper Functions

We define four helper functions used throughout the pipeline. `parseArgs` defines and parses the command line options.

⟨*Functions*⟩≡
```
func parseArgs() config {
  ⟨Parse arguments⟩
}
⟨Helper functions⟩
⟨Pipeline function⟩
```

The three required flags are `-t` for the query file, `-n` for the neighbor database, and `-d` for the SCOP database. The optional flags `-f`, `-c`, and `-r` control verbosity, early stopping, and resuming after tree construction, respectively. The flags `-c` and `-r` are mutually exclusive.

⟨*Parse arguments*⟩≡
```
var cfg config
flag.StringVar(&cfg.queryFile, "t", "", "Query file (one target species per line) [requ
flag.StringVar(&cfg.neiDB, "n", "", "Genome database path for taxi and neighbors [requi
flag.StringVar(&cfg.scopDB, "d", "", "NCBI nt/SCOP database path for scop [required]")
flag.BoolVar(&cfg.verbose, "f", false, "Feedback to terminal (quiet by default)")
flag.BoolVar(&cfg.checkOnly, "c", false, "Run up to (and including) tree plotting, then
flag.BoolVar(&cfg.resume, "r", false, "Resume after tree plotting (expects prior output
flag.Usage = func() {
  fmt.Fprintf(os.Stderr, "Usage:
" +
    "  primUR -t <query.txt> -n <neidb_path> -d <scop_db_path> [options]

" +
    "Options:
")
  flag.PrintDefaults()
}
flag.Parse()
if cfg.checkOnly && cfg.resume {
  fmt.Fprintln(os.Stderr, "Error: choose either -c or -r, not both.")
  os.Exit(1)
}
if cfg.queryFile == "" || cfg.neiDB == "" || cfg.scopDB == "" {
  fmt.Fprintln(os.Stderr, "Error: -t, -n, and -d are all required.")
  flag.Usage()
  os.Exit(1)
}
return cfg
```

`checkDependencies` verifies that all required tools are available in `PATH` using `exec.LookPath`, which is the Go equivalent of the shell's `command -v`.

⟨*Helper functions*⟩≡
```
func checkDependencies(tools []string) error {
  var missing []string
  for _, tool := range tools {
    if _, err := exec.LookPath(tool); err != nil {
      missing = append(missing, tool)
    }
  }
  if len(missing) > 0 {
    return fmt.Errorf("missing dependencies: %s",
      strings.Join(missing, ", "))
  }
  return nil
}
```

`readLines` opens a file and returns its non-empty lines as a slice of strings, using a `bufio.Scanner` for efficient line-by-line reading.

⟨*Helper functions*⟩+≡
```
func readLines(path string) ([]string, error) {
  f, err := os.Open(path)
  if err != nil {
    return nil, err
  }
  defer f.Close()
  var lines []string
  scanner := bufio.NewScanner(f)
  for scanner.Scan() {
    line := strings.TrimSpace(scanner.Text())
    if line != "" {
      lines = append(lines, line)
    }
  }
  return lines, scanner.Err()
}
```

`setupLogDir` creates the `logs/` directory and initialises empty log files for each pipeline step.

⟨*Helper functions*⟩+≡

```
func setupLogDir() error {
  err := os.MkdirAll("logs", 0755)
  if err != nil {
    return err
  }
  for _, name := range []string{
    "logs/datasets.log",
    "logs/phylonium.log",
    "logs/fur.log",
    "logs/timer.log",
  } {
    f, err := os.Create(name)
    if err != nil {
      return err
    }
    f.Close()
  }
  return nil
}
```

`safeTarget` replaces spaces with underscores in a taxon name to produce a valid directory name, mirroring the Bash substitution `${target// /_}`. `logMsg` prints a message to standard output only when verbose mode is active. `runCmd` executes an external command and returns its standard output as a string. An optional `stdin` string is passed to the process if non-empty, enabling pipeline chaining without temporary files.

⟨*Helper functions*⟩+≡

```
func safeTarget(name string) string {
  return strings.ReplaceAll(name, " ", "_")
}
func logMsg(cfg config, msg string) {
  if cfg.verbose {
    fmt.Println(msg)
  }
}
func runCmd(stdin string, name string, args ...string) (string, error) {
  cmd := exec.Command(name, args...)
  if stdin != "" {
    cmd.Stdin = strings.NewReader(stdin)
  }
  var stdout, stderr strings.Builder
  cmd.Stdout = &stdout
  cmd.Stderr = &stderr
  err := cmd.Run()
  if err != nil {
    return "", fmt.Errorf("%s failed: %w
Output: %s",
      name, err, stderr.String())
  }
  return stdout.String(), nil
}
```

Pipeline

The function `processTarget` runs the full six-step pipeline for a single target species. Output files are organised under a directory named after the target, with spaces replaced by underscores:

```
<target>/
taxids.txt
fur/        taccs.txt  naccs.txt  URs.fasta  summary.txt  fur.db
genomes/    targets/   neighbors/
primers/    primers.fasta  best_primers.fasta  scop.out
phylogeny/  tree.dist  tree.nwk  tree_clean.nwk
```

⟨*Pipeline function*⟩≡

```
  func processTarget(target string, cfg config) error {
    safe := safeTarget(target)
    logMsg(cfg, target+":")
    ⟨Create directories⟩
    if !cfg.resume {
      ⟨Step 1: Taxon lookup and neighbor discovery⟩
      ⟨Step 2: Genome download⟩
      ⟨Step 3: Phylogenetic tree⟩
    } else {
      logMsg(cfg, "  --resume set; starting at marker discovery for "+target)
    }
    if cfg.checkOnly {
      logMsg(cfg, "  --check set; stopping after tree plotting for "+target)
      return nil
    }
    ⟨Step 4: FUR marker discovery⟩
    ⟨Step 5: Primer design⟩
    ⟨Step 6: In silico specificity test⟩
    logMsg(cfg, "Finished run for "+safe+"
  ")
    return nil
  }
```

All output directories are created upfront with `os.MkdirAll`, which creates parent directories as needed.

⟨*Create directories*⟩≡

```
for _, dir := range []string{
  safe + "/fur",
  safe + "/genomes/targets",
  safe + "/genomes/neighbors",
  safe + "/primers",
  safe + "/phylogeny",
} {
  if err := os.MkdirAll(dir, 0755); err != nil {
    return err
  }
}
```

Step 1: Taxon Lookup and Neighbor Discovery

We call `taxi` to resolve the target species name to a taxon ID, then pass that ID to `neighbors` to find related genome accessions. The taxon ID is written to `taxids.txt` for later use by `scop`. Target and neighbor accessions are written to `fur/taccs.txt` and `fur/naccs.txt` respectively.

⟨*Step 1: Taxon lookup and neighbor discovery*⟩≡

```
taxiOut, err := runCmd("", "taxi", target, cfg.neiDB)
if err != nil {
  return err
}
lines := strings.Split(taxiOut, "
")
fields := strings.Fields(lines[1])
taxonID := fields[0]
err = os.WriteFile(safe+"/taxids.txt", []byte(taxonID+"
"), 0644)
if err != nil {
  return err
}
neighOut, err := runCmd("", "neighbors", "-g", "-l", "-t", taxonID, cfg.neiDB)
if err != nil {
  return err
}
var targets, neighbors []string
for _, line := range strings.Split(neighOut, "
") {
  fields := strings.Fields(line)
  if len(fields) < 2 {
    continue
  }
  switch fields[0] {
  case "t":
    targets = append(targets, fields[1])
  case "n":
    neighbors = append(neighbors, fields[1])
  }
}
err = os.WriteFile(safe+"/fur/taccs.txt",
  []byte(strings.Join(targets, "
")+"
"), 0644)
if err != nil {
  return err
}
err = os.WriteFile(safe+"/fur/naccs.txt",
```

```
    []byte(strings.Join(neighbors, "
  ")+"
  "), 0644)
  if err != nil {
    return err
  }
  logMsg(cfg, "  taxi and neighbors complete")
```

Step 2: Genome Download

For each accession, we download a ZIP archive from NCBI using `datasets`, unzip it into a temporary directory, move the `.fna` file to the appropriate genomes subfolder with a prefix (`t_` for targets, `n_` for neighbors), and delete the ZIP. The temporary directory is removed after each loop.

⟨*Step 2: Genome download*⟩≡

```
err = os.MkdirAll(safe+"/genomes/targets/tmp_unzip", 0755)
if err != nil {
  return err
}
err = os.MkdirAll(safe+"/genomes/neighbors/tmp_unzip", 0755)
if err != nil {
  return err
}
for _, taccs := range targets {
  _, err := runCmd("", "datasets", "download", "genome",
    "accession", taccs,
    "--filename", safe+"/genomes/targets/"+taccs+".zip")
  if err != nil {
    return err
  }
  logMsg(cfg, "  Downloaded: "+taccs)
  _, err = runCmd("", "unzip", "-o",
    safe+"/genomes/targets/"+taccs+".zip",
    "-d", safe+"/genomes/targets/tmp_unzip")
  if err != nil {
    return err
  }
  os.Remove(safe + "/genomes/targets/" + taccs + ".zip")
  filepath.Walk(safe+"/genomes/targets/tmp_unzip",
    func(path string, info os.FileInfo, err error) error {
      if filepath.Ext(path) == ".fna" {
        dest := safe + "/genomes/targets/t_" + filepath.Base(path)
        return os.Rename(path, dest)
      }
      return nil
    })
}
os.RemoveAll(safe + "/genomes/targets/tmp_unzip")
for _, naccs := range neighbors {
  _, err := runCmd("", "datasets", "download", "genome",
    "accession", naccs,
    "--filename", safe+"/genomes/neighbors/"+naccs+".zip")
  if err != nil {
    return err
```

```
    }
    logMsg(cfg, "  Downloaded: "+naccs)
    _, err = runCmd("", "unzip", "-o",
      safe+"/genomes/neighbors/"+naccs+".zip",
      "-d", safe+"/genomes/neighbors/tmp_unzip")
    if err != nil {
      return err
    }
    os.Remove(safe + "/genomes/neighbors/" + naccs + ".zip")
    filepath.Walk(safe+"/genomes/neighbors/tmp_unzip",
      func(path string, info os.FileInfo, err error) error {
        if filepath.Ext(path) == ".fna" {
          dest := safe + "/genomes/neighbors/n_" + filepath.Base(path)
          return os.Rename(path, dest)
        }
        return nil
      })
  }
  os.RemoveAll(safe + "/genomes/neighbors/tmp_unzip")
  logMsg(cfg, "  All genome downloads complete")
```

Step 3: Phylogenetic Tree

We compute a pairwise distance matrix with `phylonium` across all target and neighbor genomes. Because `phylonium` writes its output to standard output but also emits warnings to standard error, we redirect standard output directly to `tree.dist` and ignore the exit status. The distance matrix is then passed through `nj` to construct a neighbor-joining tree, rooted by `midRoot`, and annotated by `land`. Accession labels are cleaned with a regular expression to retain only the `GCA`/`GCF` accession number. The final tree is visualised with `plotTree`.

⟨*Step 3: Phylogenetic tree*⟩≡

```
targetFnas, err := filepath.Glob(safe + "/genomes/targets/*.fna")
if err != nil {
  return err
}
neighborFnas, err := filepath.Glob(safe + "/genomes/neighbors/*.fna")
if err != nil {
  return err
}
allFnas := append(targetFnas, neighborFnas...)
distFile, err := os.Create(safe + "/phylogeny/tree.dist")
if err != nil {
  return err
}
var phyStderr strings.Builder
cmd := exec.Command("phylonium", allFnas...)
cmd.Stdout = distFile
cmd.Stderr = &phyStderr
cmd.Run()
distFile.Close()
logMsg(cfg, "  phylonium complete")
njOut, err := runCmd("", "nj", safe+"/phylogeny/tree.dist")
if err != nil {
  return err
}
rootOut, err := runCmd(njOut, "midRoot")
if err != nil {
  return err
}
landOut, err := runCmd(rootOut, "land")
if err != nil {
  return err
}
err = os.WriteFile(safe+"/phylogeny/tree.nwk", []byte(landOut), 0644)
if err != nil {
  return err
}
```

```
treeNwk, err := os.ReadFile(safe + "/phylogeny/tree.nwk")
if err != nil {
  return err
}
step1 := strings.ReplaceAll(string(treeNwk), "'", "''")
re := regexp.MustCompile(`(GC[AF]_[0-9]+\.[0-9]+)[^']*`)
step2 := re.ReplaceAllString(step1, "$1")
err = os.WriteFile(safe+"/phylogeny/tree_clean.nwk", []byte(step2), 0644)
if err != nil {
  return err
}
_, err = runCmd("", "plotTree", safe+"/phylogeny/tree_clean.nwk")
if err != nil {
  return err
}
logMsg(cfg, "  Tree construction complete")
```

Step 4: Unique Region Discovery

`makeFurDb` builds a database from the target and neighbor genome directories. `fur` then identifies sequences present in all targets but absent from all neighbors. The output is cleaned with `cleanSeq` and written to `fur/URs.fasta`. A summary of the unique regions is produced by `cres` and written to `fur/summary.txt`.

⟨*Step 4: FUR marker discovery*⟩≡

```
_, err := runCmd("", "makeFurDb",
  "-t", safe+"/genomes/targets",
  "-n", safe+"/genomes/neighbors",
  "-d", safe+"/fur/fur.db")
if err != nil {
  return err
}
furOut, err := runCmd("", "fur", "-d", safe+"/fur/fur.db")
if err != nil {
  return err
}
furFasta, err := runCmd(furOut, "cleanSeq")
if err != nil {
  return err
}
cresOut, err := runCmd(furFasta, "cres")
if err != nil {
  return err
}
err = os.WriteFile(safe+"/fur/URs.fasta", []byte(furFasta), 0644)
if err != nil {
  return err
}
err = os.WriteFile(safe+"/fur/summary.txt", []byte(cresOut), 0644)
if err != nil {
  return err
}
logMsg(cfg, "  fur analysis completed")
```

Step 5: Primer Design

`fa2prim` converts the unique regions to primer3 input format. `primer3_core` designs candidate primers, and `prim2tab` reformats the output into a tab-separated table with columns for penalty, forward sequence, reverse sequence, and internal oligo. We parse this table, sort the rows by penalty in ascending order using an insertion sort, and write all pairs with penalty ≤ 1 to `primers/primers.fasta`. The best pair (lowest penalty) is written separately to `primers/best_primers.fasta` for use in the specificity test.

⟨*Step 5: Primer design*⟩≡

```
fa2primOut, err := runCmd("", "fa2prim", safe+"/fur/URs.fasta")
if err != nil {
  return err
}
prim3Out, err := runCmd(fa2primOut, "primer3_core")
if err != nil {
  return err
}
prim2tabOut, err := runCmd(prim3Out, "prim2tab")
if err != nil {
  return err
}
type primerRow struct {
  penalty float64
  forward string
  reverse string
}
var rows []primerRow
for _, line := range strings.Split(prim2tabOut, "
") {
  if line == "" || strings.HasPrefix(line, "#") {
    continue
  }
  fields := strings.Fields(line)
  if len(fields) < 3 {
    continue
  }
  var p float64
  fmt.Sscanf(fields[0], "%f", &p)
  rows = append(rows, primerRow{p, fields[1], fields[2]})
}
if len(rows) == 0 {
  logMsg(cfg, "  No primers found for "+target)
  return nil
}
for i := 1; i < len(rows); i++ {
```

```
    for j := i; j > 0 && rows[j].penalty < rows[j-1].penalty; j-- {
      rows[j], rows[j-1] = rows[j-1], rows[j]
    }
  }
  var fastaLines []string
  for _, r := range rows {
    if r.penalty > 1.0 {
      break
    }
    fastaLines = append(fastaLines,
      fmt.Sprintf(">f penalty: %f", r.penalty),
      r.forward,
      ">r",
      r.reverse,
    )
  }
  err = os.WriteFile(safe+"/primers/primers.fasta",
    []byte(strings.Join(fastaLines, "
")+"
"), 0644)
  if err != nil {
    return err
  }
  best := rows[0]
  bestFasta := fmt.Sprintf(">f penalty: %f
%s
>r
%s
",
    best.penalty, best.forward, best.reverse)
  err = os.WriteFile(safe+"/primers/best_primers.fasta",
    []byte(bestFasta), 0644)
  if err != nil {
    return err
  }
  logMsg(cfg, "  primer3 complete")
```

Step 6: *In Silico* Specificity Test

`scop` scores the best primer pair against a BLAST nucleotide database, using the taxon IDs in `taxids.txt` to define true positives. It reports sensitivity, specificity, and lists of true positives, false positives, and false negatives. The output is written to `primers/scop.out`.

⟨*Step 6: In silico specificity test*⟩≡

```
scopOut, err := runCmd("", "scop",
  "-d", cfg.scopDB,
  "-t", safe+"/taxids.txt",
  safe+"/primers/best_primers.fasta")
if err != nil {
  return err
}
err = os.WriteFile(safe+"/primers/scop.out", []byte(scopOut), 0644)
if err != nil {
  return err
}
logMsg(cfg, "  scop complete")
```